

## Day 2 — ADO Usage (Hands-On Workshop)

**Activity packet** for participant triads. Today's job: load the feature into Azure DevOps with disciplined hierarchy, fields, and tags. Build the queries that surface state, blocked work, and flow. Draft DP §2.

### Where we are in the week

---

The outcome map (DP §1) tells us **what we're trying to produce**. Today's ADO work tells us **how we're tracking the producing**. By 16:00, every triad's feature is loaded into ADO with a hierarchy that an engineering team could pick up Monday.

### Inputs

---

- DP §1 (outcome map)
  - PRD §6 AC
  - TCD §5, §6 (trade-offs + sign-off matrix)
  - TMD §§1–5 (data, cloud, API, sequences, baselines)
  - SEP §5 (negotiated outcomes — what survived)
-

## ADO vocabulary card (today's reference)

| TERM                       | WHAT IT IS                                               | TPM-RELEVANT                                             |
|----------------------------|----------------------------------------------------------|----------------------------------------------------------|
| <b>Work item</b>           | The atomic unit (Epic, Feature, User Story, Task, Bug)   | Everything tracked is one of these                       |
| <b>Hierarchy</b>           | Epic → Feature → User Story → Task                       | The default in Agile process; configurable               |
| <b>Area path</b>           | Logical product area ( Reconcile / Mobile )              | Filters reports                                          |
| <b>Iteration path</b>      | Sprint number / time window                              | When the work is scheduled                               |
| <b>State</b>               | New / Active / Resolved / Closed (varies by template)    | Workflow status                                          |
| <b>Tags</b>                | Free-form labels                                         | Cross-cutting filters ( security-review , customer-XYZ ) |
| <b>Story points</b>        | Effort estimate (typically Fibonacci: 1, 2, 3, 5, 8, 13) | Velocity input                                           |
| <b>Acceptance Criteria</b> | The criteria from PRD §6, attached to the work item      | Definition of done                                       |
| <b>Query (WIQL)</b>        | A saved filter                                           | "Show me everything in this state / iteration / tag"     |
| <b>Board</b>               | Kanban view of work items                                | Visual flow                                              |
| <b>Backlog</b>             | Prioritized list view                                    | Planning view                                            |
| <b>Sprint board</b>        | Filtered Kanban for current iteration                    | Daily-standup view                                       |

## The four-level hierarchy (today's spine)

```
Epic – strategic theme; usually 1 per quarterly initiative
├─ Feature – a deliverable slice; usually 1–3 per Epic
│   └─ User Story – a unit of user-visible value
│       └─ Task – engineering / design / TPM work to deliver
│           └─ User Story
│               └─ Task
└─ Feature
```

For a single feature in the academy's scope:

- **1 Epic** for the broader theme it sits in (e.g., "Reduce dispatcher after-shift admin load")
- **1 Feature** for this PRD ("Reconcile flow")
- **5–10 User Stories** (each Acceptance Criterion roughly maps to one)
- **2–4 Tasks per User Story**

The discipline: **don't go deeper than 4 levels** (no sub-tasks of sub-tasks). Don't promote a Task to a User Story to "make it visible." User Stories have user-visible value; tasks don't.

## Field discipline (the part most teams skip)

---

Each work item has fields. Most teams populate the title and skip the rest. Mature teams populate:

| FIELD               | WHY IT MATTERS                      |
|---------------------|-------------------------------------|
| Title               | Clear, action-oriented              |
| Description         | One paragraph context               |
| Acceptance Criteria | Pulled from PRD §6; testable        |
| Story Points        | Effort estimate (User Stories only) |
| State               | Workflow visibility                 |
| Area path           | Reporting hierarchy                 |
| Iteration path      | Sprint scheduling                   |
| Tags                | Cross-cutting filters               |
| Linked work items   | Parent / child / related            |
| Assignee            | Who's currently working on it       |

The cost of skipping fields shows up at sprint review when nobody can find anything. The cost of populating them is ~5 minutes per item.

---

## Activity 1 — Build the Hierarchy

---

**Format:** Triad • 35 min • Block 1

### Purpose

Map the feature into the Epic → Feature → User Story → Task hierarchy.

### Triad protocol

#### Step 1 — Define the Epic (5 min)

```
Title: <strategic theme – usually 1 per quarter>
Description: <2-paragraph context: business goal + customer outcome>
Tags: <area, OKR identifier>
```

For FieldPulse: "Reduce dispatcher after-shift admin load" — the umbrella for all reconcile work, manager-view, etc.

#### Step 2 — Define the Feature (5 min)

```
Title: <the PRD's feature name>
Description: <1-paragraph from PRD §1>
Parent: <link to the Epic>
Tags: <release tag, persona tag>
```

For FieldPulse: "Reconcile flow (mobile)".

### Step 3 — Generate User Stories (15 min)

For each PRD §6 Acceptance Criterion (or related cluster), draft a User Story:

```
Title: As a <persona>, I want <capability> so that <outcome>
Description: <1-paragraph from PRD §5 sketch>
Acceptance Criteria: <copy from PRD §6, possibly grouped>
Story Points: <Fibonacci estimate>
Parent: <link to the Feature>
Tags: <happy-path / sad-path / weird-path / non-functional>
```

Aim for **5–10 User Stories** per Feature.

### Step 4 — Tag NFRs as separate stories (10 min)

NFRs from PRD §7 / TCD §3 / §4 don't fit cleanly as user stories. Common treatments:

- Performance NFR → its own User Story tagged `non-functional` ("As a dispatcher, I want the modal to open in <1s...")
- Security NFR → a Task on the relevant User Story OR its own `security` story
- Compliance NFR → its own `compliance` story

The discipline: **NFRs are work**. They go in the backlog or they don't get done.

## Output

A skeleton: 1 Epic + 1 Feature + 5–10 User Stories with parent links.

---

## Activity 2 — Field Discipline + Tasks

---

**Format:** Triad • 40 min • Block 2

### Purpose

Populate fields on every work item. Add Tasks under each User Story.

### Triad protocol

#### Step 1 — Field pass on every work item (15 min)

For each Epic / Feature / User Story:

- ☐ Title is action-oriented and specific
- ☐ Description has 1 paragraph of context
- ☐ Acceptance Criteria populated (User Stories) or empty (Epic / Feature)
- ☐ Area path set (e.g., `Dispatcher Workflow / Mobile`)
- ☐ Iteration path set (assign to a sprint, even if estimated)
- ☐ State set ( `New` for everything until work begins)
- ☐ Tags appropriate

#### Step 2 — Story-point estimation (10 min)

For each User Story, assign Fibonacci points (1, 2, 3, 5, 8, 13). Use **planning poker** if possible — each triad member estimates silently, then reveals. Discuss disparities. Don't average.

If a story is **>13 points**, break it into 2+ smaller stories. >13 means it's too vague to estimate.

### Step 3 — Task breakdown (15 min)

For each User Story, draft 2–4 Tasks:

```
Title: <verb-first: "Build", "Test", "Document", "Review">
Description: <1 line>
State: New
Parent: <link to User Story>
Assignee: <if known, otherwise empty>
```

Tasks for a typical user story:

- "Build" — the implementation work
- "Test" — automated test coverage
- "Document" — release notes, internal docs
- "Review" — security review, peer review, stakeholder sign-off

### What "good" looks like

- Every work item has all required fields populated
- Story points are **specific**, not "we don't know"
- Tasks are **verb-first** and **specific** ("Build Reconcile API endpoint" not "API work")
- Stories with >13 points are split

---

## Activity 3 — Queries (WIQL) + Boards

Format: Triad • 40 min • Block 3

### Purpose

Build the queries that surface what's happening. Set up the boards for sprint visibility.

### The 5 queries every team needs

#### 1. "What's open in the current sprint?"

```
Type: User Story or Task
State: New / Active
Iteration Path: Current
Area Path: Under <our area>
```

Used: every standup.

#### 2. "What's blocked?"

```
Type: User Story or Task
State: Active
Tags: Contains "blocked"
```

Used: daily; surfaces dependencies that are stuck.

### 3. "What's done in this sprint?"

```
Type: User Story  
State: Closed  
Iteration Path: Current  
Area Path: Under <our area>
```

Used: end-of-sprint review; outcome verification.

### 4. "What hasn't been touched in N days?"

```
Type: User Story or Task  
State: Active  
Changed Date: < (Today - 7 days)
```

Used: weekly; catches forgotten work.

### 5. "What's in this NFR / quality category?"

```
Type: User Story  
Tags: Contains "non-functional" OR "security" OR "compliance"  
State: Not Closed
```

Used: monthly; ensures NFRs aren't being deferred indefinitely.

## Triad protocol

1. **Build all 5 queries** in ADO (15 min). Save them in a shared folder.
2. **Set up the sprint board** (10 min). Configure columns: To Do / In Progress / In Review / Done. Add WIP limits where appropriate (e.g., max 3 In Progress per developer).
3. **Set up the Kanban board** for the Feature (10 min). Same columns; this view is for the whole feature, not just one sprint.
4. **The "what would I check daily?"** (5 min). Each member identifies the one query they'd run every morning. Discuss; converge.

## What "good" looks like

- All 5 queries are saved and named clearly
- The sprint board has columns matching the team's actual flow (not just default)
- WIP limits are set explicitly — they're a forcing function

---

## Activity 4 — Reading the Charts + AI as Standup Aid

---

**Format:** Triad • 45 min + Wrap • Block 4

### Purpose

Read the standard ADO charts (cumulative flow, burn-down, velocity) and learn to use AI as a standup aid (without replacing the team).

## Reading the charts

### Cumulative Flow Diagram (CFD)

A stacked area chart showing how many items are in each state over time. Used for:

- **Spotting WIP bloat** — if "In Progress" expands faster than "Done", work is starting but not finishing
- **Spotting bottlenecks** — if a state's band is widening, items are queuing there
- **Estimating cycle time** — horizontal distance from "To Do" entry to "Done" entry

### Burn-down chart

How many points / items remain in the current sprint. Used for:

- **Mid-sprint sanity check** — is the team on track to complete by end of sprint?
- **Identifying scope creep** — does the line jump up mid-sprint?

### Velocity chart

Story points completed per sprint, over time. Used for:

- **Capacity planning** — average velocity is your input to next-sprint commitment
- **Spotting team-health issues** — declining velocity often signals burnout, blockers, or context-switching

## AI as standup aid

A useful pattern: AI summarizes the past day's ADO activity for the standup, surfaces anomalies, drafts the "what's blocking us" framing.

```
Role: Standup-prep assistant for a TPM.
Context: <paste yesterday's ADO query results — items moved
        state, items added, items closed, comments added>
Task: Produce a 5-bullet standup summary:
  1. Top 2 things shipped
  2. Top 2 things blocked or stuck (>2 days no movement)
  3. 1 anomaly (pattern that's unusual for this team)
Constraints:
  - Only use information in the provided data
  - Flag anything you can't determine
Format: 5 numbered bullets. Each bullet under 15 words.
```

Validation discipline: same as Week 5 Day 5. Cross-check the AI's claims against the actual ADO data.

## Triad protocol

1. **Walk the three charts** (15 min). For each, identify what your team would look for.
2. **Run the AI standup-prep prompt** (10 min). Use yesterday's ADO state (or a synthetic example).
3. **Validate the output** (10 min). What's wrong? What's right? What's missing?
4. **Document in DP §2** (10 min). The 5 queries, the board configuration, the AI-aid pattern — all referenced.

## Wrap (last 15 min)

Each triad shares:

- The **one query** they'd run every morning
  - The **WIP limit** they set and why
  - One **failure mode** in their hierarchy / fields they expect to surface in a real sprint review
-

## End-of-day checkpoint

---

Each triad ends Day 2 with:

- ✓ **Loaded ADO** (or paper backlog): 1 Epic + 1 Feature + 5–10 User Stories + 2–4 Tasks each
- ✓ All work items have **Title / Description / AC / Area / Iteration / State / Tags**
- ✓ **Story points** estimated; stories >13 split
- ✓ **5 saved queries** for state, blocked, done, stale, NFR
- ✓ **Sprint board** + **Kanban board** configured with WIP limits
- ✓ AI provenance entry (standup-prep)
- ✓ DP §2 drafted with links / screenshots